Arduino Group Project - Group 9
Module code: DG1FDS

Team Members
Khamis Mohamed - 250179000
Zainab Suleiman Pai - 240174569
Malique Cummings -250262379
Alliyah Mbugua – 240376118
Jarnail Sehjal –210168660

Saif Ali- 240153047 (Did not attend any labs, the group never met him) Contribution mark is 0%

Contribution Table

Student ID	Student Name	Role and Responsibilities	Agreed contribution
250179000	Khamis Mohamed	Hardware specialist: Led the task 1 weather station coding , integrated the potentiometer and OLED for Task 2, and implemented the countdown timer for Task 3.	100%
240174569	Zainab Suleiman Pai	Responsible for software: developed the logic for the night-light (Task 2) and pedestrian crossing (Task 3).	100%
240376118	Alliyah Mbugua	Responsible for the report wrote the explanations, formatted the document, tested the system, and submitted the work.	100%
210168660	Jarnail Sehjal	Calibrated sensors for Task 1, ensured correct pressure units (hPa), and added OLED bar graph visualization.	100%
250262379	Malique Cummings	Task 1 Buzzer, OLED displays for task 1, Task 3 Return to Stop with RED LED, RED LED to Stop.	100%

This is the code for the first task, and the needed libraries are grove temp and humidity sensor, u8g2 and the seeed_BMP280.

Task 1: Mini Weather station COMPLETE TASK

```
#include <Wire.h>
#include <DHT20.h>   // Correct library for DHT20
#include "Seeed_BMP280.h"
#include <U8x8lib.h>
```

```
// DHT20
#define BUZZER_PIN 5
#define POT_PIN A0
DHT20 dht;   // DHT20 uses this constructor
// BMP280
BMP280 bmp280;
// OLED

U8X8_SSD1306_128X64_NONAME_HW_I2C u8x8(/* reset=*/ U8X8_PIN_NONE);
//VARIABLES
float temperatureThreshold = 25.0;          //the temperature is default threshold
int lastPotValue = 0;
void setup() {
// Initialise Serial Monitor
Serial.begin(115200);
Wire.begin();

// Initialise DHT20
dht.begin();
Serial.println("DHT20 sensor initialised");     // this is to send a message to the pc (for debugging and monitoring)

//bmp280
if (!bmp280.init()) {
   Serial.println("BMP280 not connected or broken!");
} else {
   Serial.println("BMP280 sensor initialised");
}

// Initialise OLED
u8x8.begin();
u8x8.setPowerSave(0);
u8x8.setFlipMode(1);
Serial.println("OLED display initialised");

//starting buzzer
pinMode(BUZZER_PIN, OUTPUT);
digitalWrite(BUZZER_PIN, LOW);
Serial.println("Buzzer initialised on pin D5");
Serial.println("Weather Station Ready!\n");
delay(1000);
}
void loop() {
int potValue = analogRead(POT_PIN);

if (abs(potValue - lastPotValue) > 10) { // Report only when substantial changes occur.
   temperatureThreshold = map(potValue, 0, 1023, 150, 350) / 10.0;
   lastPotValue = potValue;
   Serial.print("Threshold adjust to: ");
   Serial.print(temperatureThreshold);
   Serial.println("°C");
```

```
}
// --- DHT20 Sensor ---
dht.read();
float dhtTemp = dht.getTemperature();
float dhtHumi = dht.getHumidity();
// --- BMP280 Sensor ---
float pressure = bmp280.getPressure(); // Pa
//  Print to Serial
Serial.print("DHT Temp: "); Serial.print(dhtTemp); Serial.println(" C");
Serial.print("DHT Humidity: "); Serial.print(dhtHumi); Serial.println(" %");
Serial.print("Pressure: "); Serial.print(pressure); Serial.println(" Pa");
Serial.println();

// --- Display on OLED ---
u8x8.setFont(u8x8_font_chroma48medium8_r);
u8x8.setCursor(0,0);
u8x8.print("Temp-Level:"); u8x8.print(dhtTemp,1); u8x8.print("C");
u8x8.setCursor(0,1);
u8x8.print("Hum-Level:"); u8x8.print(dhtHumi,1); u8x8.print("%");
u8x8.setCursor(0,3);
u8x8.print("Pressure:"); u8x8.print(pressure/100.0,1); u8x8.print("hPa");
//CHALLENGE 2: BAR GRAPH AND THRESHOLD DISPLAY
//showing the current threshold value on row 5
u8x8.setCursor(0, 5);
u8x8.print("Thresh:"); u8x8.print(temperatureThreshold, 1); u8x8.print("C");

int barLength = map(temperatureThreshold, 15, 35, 0, 16);
u8x8.setCursor(0, 6);
u8x8.print("[");
for (int i = 0; i < barLength; i++) u8x8.print("=");
for (int i = barLength; i < 16; i++) u8x8.print(" ");
u8x8.print("]");
u8x8.refreshDisplay();
//BUZZER CONTROL
if (dhtTemp > temperatureThreshold) {
   digitalWrite(BUZZER_PIN, HIGH);
} else {
   digitalWrite(BUZZER_PIN, LOW);
}
delay(1000); // Wait 1 second before next reading
}
```

Task 2: Smart Night-Light System

This is the code for task 2, and the needed libraries is only the u8g2.

```
#include <Wire.h>
#include <U8g2lib.h>
U8G2_SSD1306_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0);

const int ledPin = 4;
const int lightPin = A6;
const int potPin = A0;

int lightValue = 0;
int threshold = 0;

void setup() {
  pinMode(ledPin, OUTPUT);
  Serial.begin(600);
  u8g2.begin();
}

void loop() {
  // Read sensors
  lightValue = analogRead(lightPin);
  threshold = analogRead(potPin);
  // LED control
  if(lightValue < threshold){
    digitalWrite(ledPin, HIGH);
  }
  else{
    digitalWrite(ledPin, LOW);
  }
  // OLED Display
  u8g2.clearBuffer();
  u8g2.setFont(u8g2_font_ncenB08_tr);

  u8g2.drawStr(0,15,"Light Level:");
  u8g2.setCursor(90,15);
  u8g2.print(lightValue);
  u8g2.drawStr(0,35,"Threshold:");
  u8g2.setCursor(90,35);
  u8g2.print(threshold);

  if(lightValue < threshold){
    u8g2.drawStr(0,55,"LED: ON (Dark)");
  }
```

```
  else{
    u8g2.drawStr(0,55,"LED: OFF (Bright)");
  }
  u8g2.sendBuffer();
  delay(200);
}
```

Task 3 with challenges : pedestrian crossing

```
#include <Wire.h>
#include <U8x8lib.h>
//=====PIN DEFINITIONS=====
// I GOT THIS FROM WEEK 6 LAB 1, FROM BROVE BEGINNER KIT DOCUMENTATION
#define RED_LED_PIN 4        // red light on D4
#define BUTTON_PIN 6         // button on D6
#define BUZZER_PIN 5         // buzzer on D5 (from lab 3)
//---------- CHALLENGE 2: POTENTIOMETER PIN----------
#define POT_PIN A0
//=====OLED DISPLAY=====
// Using Option A: SSD1306 128x64 I2C OLED
U8X8_SSD1306_128X64_NONAME_HW_I2C u8x8(U8X8_PIN_NONE);

//=====STATE VARIABLES=====
enum SystemState {
  STATE_STOP,          // When the red light is on, display "STOP"
  STATE_COUNTDOWN,     // This counts down before GO shows up
  STATE_GO,            // Crossing time, display "GO"
  STATE_WARNING        // CHALLENGE 1: flashing LED before returning to STOP
};

SystemState currentState = STATE_STOP;
int countdown = 10;          // starts to countdown from 10 seconds
int crossingTime = 10;       // crossing time (GO) in 10 seconds
bool buttonPressed = false;  // tracks button press
unsigned long lastUpdate = 0; // for timing
//----------CHALLENGE 1: FLASHING LED VARIABLES----------
bool ledState = false;
unsigned long finalFlash = 0; // for timing
//----------CHALLENGE 2: ADJUSTABLE VARIABLES----------
int baseCountdown = 10;
int baseCrossingTime = 10;

void setup() {
  Serial.begin(9600);
  Serial.println("Task 3: Pedestrian Crossing System ACTIVE!");
  // INITIALISE I2C BUS
  Wire.begin();
```

```
  // INITIALISE OLED DISPLAY (from week 8 lab)
  u8x8.begin();
  u8x8.setPowerSave(0);
  u8x8.setFlipMode(1);
  // INITIALISE PINS (from Lab 1)
  pinMode(RED_LED_PIN, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);  // button press gives LOW
  pinMode(BUZZER_PIN, OUTPUT);        // buzzer setup (Lab 3)
  // Starts with all outputs OFF
  digitalWrite(RED_LED_PIN, HIGH);    // Red LED is ON (HIGH = active)
  digitalWrite(BUZZER_PIN, LOW);      // Buzzer OFF
  // Display initial STOP message
  u8x8.setFont(u8x8_font_chroma48medium8_r);
  u8x8.setCursor(0, 2);
  u8x8.print("   STOP   ");
  u8x8.setCursor(0, 4);
  u8x8.print("PRESS BUTTON");
  Serial.println("System Ready – Push to Go");
  Serial.println("=====================");
}
void loop() {
  //----------CHALLENGE 2: ADJUSTABLE TIMERS-----------
  int potValue = analogRead(POT_PIN);
  // mapping the potentiometer to 5–15 seconds countdown time
  baseCountdown = map(potValue, 0, 1023, 5, 15);
  // showing current timer setting on OLED
  u8x8.setCursor(0, 6);
  u8x8.print("TIMER: ");
  u8x8.print(baseCountdown);
  u8x8.print("s   ");
  //========== READ BUTTON ==========
  bool buttonState = (digitalRead(BUTTON_PIN) == LOW);
  // detect button press (transition from not pressed → pressed)
  if (buttonState && !buttonPressed) {
    buttonPressed = true;
    // only trigger if system is in STOP state
    if (currentState == STATE_STOP) {
      Serial.println("Button Pressed! Watch for GO signal");
      currentState = STATE_COUNTDOWN;
      // CHALLENGE 2: USING ADJUSTABLE VALUE
      countdown = baseCountdown;
      lastUpdate = millis();
      digitalWrite(RED_LED_PIN, LOW); // turn OFF red LED during countdown
      // clear "PRESS BUTTON" message
      u8x8.setCursor(0, 4);
      u8x8.print("            ");
    }
  }
  // reset button flag when released
  if (!buttonState) {
```

```
  buttonPressed = false;
}
//========= SEQUENTIAL LOGIC==========
unsigned long now = millis();
switch (currentState) {
  case STATE_STOP:
    // Nothing to do
    break;
  case STATE_COUNTDOWN:
    if (now - lastUpdate >= 1000) {
      lastUpdate = now;
      countdown--;
      u8x8.setCursor(0, 3);
      u8x8.print("READY IN: ");
      u8x8.print(countdown);
      u8x8.print("s   ");
      Serial.print("READY IN: ");
      Serial.println(countdown);
      //========== BUZZER SECTION ==========
      if (countdown <= 0) {
        currentState = STATE_GO;
        // CHALLENGE 2: USE ADJUSTABLE VALUES
        crossingTime = baseCountdown;
        // Buzzer beep (Lab 3 style)
        Serial.println("GO! You can cross now");
        digitalWrite(BUZZER_PIN, HIGH);
        delay(700);
        digitalWrite(BUZZER_PIN, LOW);
        // Display GO message
        u8x8.setCursor(0, 2);
        u8x8.print("    GO!    ");
        lastUpdate = millis();
      }
    }
    break;
  case STATE_GO:
    if (now - lastUpdate >= 1000) {
      lastUpdate = now;
      crossingTime--;
      // show remaining crossing time on OLED
      u8x8.setCursor(0, 4);
      u8x8.print("CROSS: ");
      u8x8.print(crossingTime);
      u8x8.print("s   ");
      Serial.print("CROSS TIME: ");
      Serial.println(crossingTime);
      // when crossing duration ends, go to WARNING
      if (crossingTime <= 0) {
        currentState = STATE_WARNING;
        finalFlash = millis();
```

```
        ledState = false;
        Serial.println("GET READY TO STOP");
      }
    }
    break;
  case STATE_WARNING: {
    // flash LED three times before STOP
    static int flashCount = 0;
    if (now - finalFlash >= 500) {
      finalFlash = now;
      ledState = !ledState;
      digitalWrite(RED_LED_PIN, ledState);
      // buzzer beeps with each flash
      if (ledState) {
        digitalWrite(BUZZER_PIN, HIGH);
        delay(100);
        digitalWrite(BUZZER_PIN, LOW);
      }
      if (ledState) flashCount++;
      if (flashCount >= 6) { // 3 full on/off cycles
        flashCount = 0;
        currentState = STATE_STOP;
        digitalWrite(RED_LED_PIN, HIGH); // LED stays ON
        // display STOP message
        u8x8.setCursor(0, 2);
        u8x8.print("   STOP   ");
        u8x8.setCursor(0, 4);
        u8x8.print("PRESS BUTTON");
        Serial.println("CROSSING COMPLETE");
        Serial.println("BACK TO STOP");
      }
    }
    break;
  }
 }
 delay(60);
}
```